Prepare answers before the exercise class. In class you will present them.

4.1 Implementing Causal, FIFO-ordered and Atomic broadcast

For this exercise use our pseudo-code notation that we used in the lecture!

a) Assuming you have an abstraction of causal broadcast (`cbcast`), how can you implement reliable broadcast (`rbcast`) on this basis?

b) Assuming you have an abstraction of reliable broadcast (`rbcast`), how can you implement FIFO-broadcast (`fbcast`) on this basis?

c) Assuming you have an abstraction of atomic broadcast (`abcast`), how can you implement FIFO-broadcast (`fbcast`) on this basis?

4.2 Timed reliable broadcast

Assume you have a synchronous system: Reliable point-to-point links have a guaranteed message transmission delay of less than d, and local computations take a negligible amount of time.

Timed reliable broadcast is a variant of reliable broadcast with an additional **Real-Time Δ-Timeliness** property: There is a known constant Δ such that if a message m is broadcast at real time t, then no correct process delivers M after real time $\mathrm{t} + \Delta$.

Proof that – under the assumption of a synchronous system – the echo algorithm for rbcast implements timed reliable broadcast. You can proof this statement by calculating an upper bound for the transmission delay Δ of the rbcast.

4.3 Uniform reliable broadcast

Assume you have an asynchronous system with a *fail-stop* fault model. This means that nodes fail by crashing, and that all crashes are reliably detected by all other correct processes. We will discuss failure detectors later, but for now you can assume that if a process $\mathrm{P_i}$ fails, all correct processes will be notified about the crash by an event FAILED($\mathrm{P_i}$).

Can you find an algorithm that implements *uniform reliable broadcast* in this system model?

Optionally: Can you find an algorithm that implements *uniform reliable broadcast* without having reliable failure detection?

4.4 Atomic broadcast without crashes

Atomic broadcast guarantees that all correct processes will deliver all messages in exactly the same order.

Can you find an algorithm for atomic broadcast for an asynchronous system model *without* node failures? Describe the algorithm using our pseudo-code notation.

4.5 Causal but not reliable

Is it possible to implement a broadcast mechanisms that guarantees the *causal order* property, but not the *agreement* property of reliable broadcast?

4.6 Reliable broadcast

A fundamental property in *reliable* broadcast, the *agreement* property, states that if a message m is delivered by some correct process, then m is eventually delivered by every correct process.

Explain exactly why the following broadcast algorithm satisfies the *agreement* property!

Algorithm Reliable Broadcast

```
Uses: PerfectPointToPointLink (ppp)

upon event < Init > do:
    delivered := ∅;

upon event < RBroadcast, m > do:
    trigger < ppp.Send, self, m >;

upon event < ppp.Receive, m > do:
    if m ∉ delivered then:
        forall p ∈ P do:        /* P: set of all processes */
            trigger < ppp.Send, p, m >;
        delivered := delivered ∪ {m};
        trigger < RDeliver, m >;
```

4.7 Consensus without faults

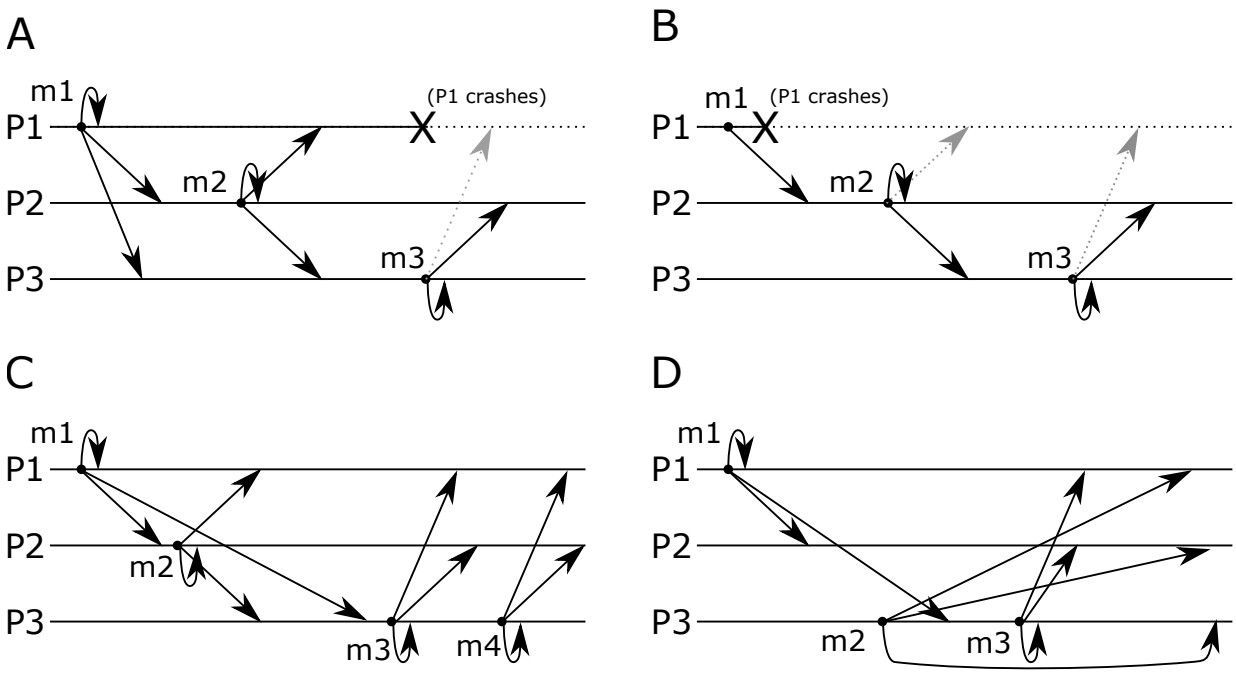
Implement a simple algorithm that solves the consensus problem in an asynchronous system with n nodes and without faults! You can use ppp (*PerfectPointToPointLink*) links between nodes.

Interface Interface and properties of (regular) consensus

```
Events:
    Request: <Propose, v>: Proposes value v for consensus.
    Indication: <Decide, v>: Outputs a decided value v of consensus.
Properties:
    • Termination: Every correct process eventually decides some
      value.
    • Validity: If a process decides v, then v was proposed by
      some process.
    • Integrity: No process decides twice.
    • Agreement: No two correct processes decide differently.
```

4.8 Comparison of broadcast semantics

Consider the following four message diagrams that show the broadcast and delivery of messages (black arrows) by a set of three processes (gray dotted arrows are messages that are not delivered because the destination process has crashed)



(a) Which of the diagrams show a possible (valid) execution of a broadcast algorithm with best effort semantics? Justify!

(b) Which of the diagrams show a possible (valid) execution of a broadcast algorithm with total order semantics (abcast)?

(c) Which of the diagrams show a possible (valid) execution of a broadcast algorithm with causal order semantics (abcast)?